

Горчак Дмитрий, 212. Отчёт: локальный репозиторий git и задача о фигурах на плоскости

Отчёт о выполненной работе: установка git, создание локального репозитория, работа с ним и текущее состояние программы. Скриншоты сняты с окон MSYS2 на моём компьютере.

Сделано:

- 1) git установлен в MSYS2 и настроен, вся работа сохраняется через локальный репозиторий ~/cones;
- 2) готова иерархия классов Point -> Circle -> Cone; фигура создаётся по количеству чисел в строке файла: два числа – точка, три – круг, четыре – конус;
- 3) площадь основания и объём реализованы методами, а не полями; поля закрыты, доступ через get/set;
- 4) объём считается отдельно, по запросу: номер нужной фигуры передаётся программе аргументом командной строки;
- 5) поле фигур нарисовано в gnuplot: вид сверху, карта высот, ландшафт и объёмный рисунок точками по поверхности фигур;
- 6) фигуры складываются в двунаправленный список STL и упорядочиваются по площади основания и по объёму;
- 7) вокруг конусов рассыпаны гауссовы точки, разбиты на блоки сетки и собраны в кластеры двумя независимыми алгоритмами;
- 8) отдельной программой проверен датчик случайных чисел: одномерное и двумерное нормальное распределение;
- 9) написаны четыре эссе – по git, по STL и двунаправленному списку, по гауссову распределению и по кластерному анализу.

Уточнение по фигуре: преподаватель подтвердил, что фигура с круглым основанием и высотой – это конус, а не пирамида. Поэтому класс называется Cone, а объём считается по формуле $V = S * h / 3 = \pi * r * r * h / 3$, где S – площадь основания, взятая у базового класса Circle. Для сравнения в классе оставлен метод volume_ngon(n) – объём пирамиды с правильным n-угольником в основании, вписанным в тот же круг: при росте n он стремится к объёму конуса.

1. Установка git в MSYS2

Git поставлен штатным менеджером пакетов MSYS2 командой `pacman -S git`. На скриншоте – запись в журнале установки, версия пакета, версия самой программы, путь к ней и начало списка команд из встроенной справки.

```
MSYS2 - git install

=== Горчак Дмитрий, 212. Установка git в MSYS2 ===

$ grep -E 'installed (git|mingw-w64-x86_64-gcc) ' /var/log/pacman.log | tail -2
[2026-05-20T00:55:46+0300] [ALPM] installed mingw-w64-x86_64-gcc (15.2.0-13)
[2026-09-17T18:16:00+0300] [ALPM] installed git (2.55.0-1)

$ pacman -Q git
git 2.55.0-1

$ git --version
git version 2.55.0

$ which git
/usr/bin/git

$ git help -a | head -8
See 'git help <command>' to read about a specific subcommand

Main Porcelain Commands
  add                Add file contents to the index
  am                 Apply a series of patches from a mailbox
  archive            Create an archive of files from a named tree
  backfill            Download missing objects in a partial clone
  bisect             Use binary search to find the commit that introduced a bug
```

Скриншот 1. Установка git и проверка: `pacman -Q git`, `git --version`, `git help -a`

2. Локальный репозиторий

Репозиторий создан прямо в папке с задачей, история ведётся по шагам работы: отдельный коммит на каждый класс, на ввод-вывод, на программу и на скрипт gnuplot.

```
$ cd ~/cones
$ git init -b main
$ git config user.name "Gorchak Dmitriy"
$ git config core.autocrlf false
```

Работа над списком из STL велась в отдельной ветке `stl-list` и влита слиянием с ключом `--no-ff`, поэтому в истории видно, что это была отдельная задача. На коммит, соответствующий сроку 19.09, поставлена метка `stage-19-09`.

```
MSYS2 - local repository

=== Горчак Дмитрий, 212. Локальный репозиторий ~/cones ===

$ cd ~/cones && ls
README.md  circle.cpp  cone.h      figures_io.cpp  main.cpp    plot_figures.gp  txt
bin         circle.h    essays      figures_io.h    make_txt.sh  point.cpp        point.h
build.sh    cone.cpp    figures.txt  list_demo.cpp  out          point.h

$ git config --list --local | grep -E 'user|autocrlf' | sed 's/\(user.email=\).*\/\1***@***/'
core.autocrlf=false
user.name=Gorchak Dmitry
user.email=***@***

$ git log --oneline --decorate | head -8
77dab7d (HEAD -> main) Плоская структура проекта; фигура по числу чисел в строке (2/3/4), объём по запросу, точки на поверхности и уравнения фигур
a8d16df Эссе переписаны в формате прошлых работ: команда-пример-пояснение и памятка в приложении
cd62534 Отчёт: обновлённые скриншоты (почта в настройках скрыта)
37e7923 Эссе по git и STL, отчёт с рисунками и скриншотами
c16383e Новый конус перенесён в раздел одиночных, ошибочная строка осталась последней
8e78f1c Ещё один конус в исходных данных
43a246a Слияние ветки stl-list: работа со списком из STL
d5efe6d (stl-list) Двухнаправленный список STL: сортировка по площади и по объёму

$ git status
On branch main
nothing to commit, working tree clean
```

Скриншот 2. Состав папки, локальные настройки, история коммитов и чистое рабочее дерево

Все исходники лежат в одной папке, без вложенных каталогов: так их проще найти и отправить. Собранные программы складываются в bin, всё, что создают программа и gnuplot, – в out; обе папки перечислены в .gitignore и в историю не попадают, поэтому git status показывает чистое дерево сразу после сборки и запуска.

3. Рабочий цикл: правка, сравнение, фиксация

Обычная работа с локальным хранилищем выглядит так: изменить файл, посмотреть, что именно изменилось, подготовить изменение к фиксации и зафиксировать его с понятным сообщением. На скриншоте в файл данных добавлен ещё один круг – строка из трёх чисел.

```
MSYS2 - status diff commit
$ echo ' 10.5  6.0  1.2' >> figures.txt

$ git status -s
M essays/screens/02-repository-log.png
M figures.txt

$ git diff
diff --git a/essays/screens/02-repository-log.png b/essays/screens/02-repository-log.png
index 292cac9..83b138a 100644
Binary files a/essays/screens/02-repository-log.png and b/essays/screens/02-repository-log.png differ
diff --git a/figures.txt b/figures.txt
index e11f1e5..2eb35b9 100644
--- a/figures.txt
+++ b/figures.txt
@@ -38,3 +38,4 @@
# строка с ошибкой: одного числа мало
12.0
+ 10.5  6.0  1.2

$ git add figures.txt

$ git commit -m 'Ещё один круг в исходных данных'
[main 72f09da] Ещё один круг в исходных данных
1 file changed, 1 insertion(+)

$ git log --oneline -3
72f09da (HEAD -> main) Ещё один круг в исходных данных
77dab7d Плоская структура проекта; фигура по числу чисел в строке (2/3/4), объём по запросу, точки на по
верхности и уравнения фигур
a8d16df Эссе переписаны в формате прошлых работ: команда-пример-пояснение и памятка в приложении
```

Скриншот 3. git status, git diff, git add, git commit и история после фиксации

Видно главное свойство diff: git показывает не файл целиком, а одну добавленную строку с отметкой + и несколько строк вокруг неё для понимания места.

4. Сборка, запуск и построение рисунков

Программы собираются одной командой `bash build.sh`: классы компилируются вместе с программой, готовые файлы складываются в папку `bin`. После запуска программа создаёт в папке `out` данные для `gnuplot`, а скрипт `plot_figures.gr` строит по ним рисунки.

```
MSYS2 - build and run
$ ./bin/figures figures.txt 9 | tail -24
18                                total:      156.45      286.89

--- Figure asked on the command line ---
figure 9: Cone(x =  9.00, y =  8.00, r =  3.00, h = 10.00, volume =  94.25)
base area = 28.27
volume    = 94.25
volume of a pyramid with the same circle around its base:
  4-gon   = 60.00
  8-gon   = 84.85
 64-gon   = 94.10

--- Field bounds ---
x: [-1.48; 22.88]
y: [-0.78; 21.18]

--- Data for gnuplot (folder 'out') ---
plan_points.txt, plan_circles.txt, plan_cones.txt -- top view
field.txt -- landscape grid 121 x 121
range.gp -- field bounds for gnuplot
surface_points.txt -- points on the surface of every figure
equations.txt -- equation of every figure

Run 'gnuplot plot_figures.gp' to draw the field.
Program finished.

$ gnuplot plot_figures.gp
plots are ready: out/plan.png, out/map.png, out/landscape.png, out/points3d.png

$ ls -l out
equations.txt
field.txt
landscape.png
map.png
plan.png
plan_circles.txt
plan_cones.txt
plan_points.txt
points3d.png
range.gp
surface_points.txt
```

Скриншот 4. Объём фигуры по запросу, файлы с данными и построение рисунков

Вторым аргументом программе передан номер фигуры – для неё печатаются площадь основания, объём и, для сравнения, объём пирамиды с правильным n -угольником в основании. Границы поля по x и y считает сама программа: она находит крайние точки оснований, добавляет запас и записывает готовые команды `gnuplot` в файл `out/range.gp`, который скрипт подключает командой `load`.

5. Успешные запуски программ

Собираются четыре программы, и каждая запускается отдельно. Первая – `figures`: она читает файл данных, печатает таблицу фигур и готовит данные для `gnuplot`. Две строки в файле сделаны намеренно неправильными (отрицательный радиус и одно число в строке); программа сообщает о них и продолжает работу, поэтому из двадцати строк получается восемнадцать объектов.

```
MSYS2 - run figures
=== Горчак Дмитрий, 212. Запуск программы figures ===
$ ./bin/figures | head -34

=====
Figures on a plane: Point -> Circle -> Cone
2 numbers make a point, 3 a circle, 4 a cone
=====

--- Class hierarchy (one call, different output) ---
Point(x = 1.00, y = 2.00)
Circle(x = 1.00, y = 2.00, r = 3.00, area = 28.27)
Cone(x = 1.00, y = 2.00, r = 3.00, h = 4.00, volume = 37.70)
a point needs 2 numbers, a circle 3, a cone 4

--- Private fields, access through get/set ---
get_r() = 2.00, get_h() = 6.00, volume() = 25.13
after set_r(4): volume() = 100.53 (the value is not stored, it is computed)
rejected: radius must be >= 0
rejected: height must be >= 0

--- Reading properties from 'figures.txt' ---
line 38 skipped: radius must be >= 0
line 41 skipped: need 2 to 4 numbers
objects created: 18

--- Stored properties and computed attributes ---
  N    type    x    y    r    h    area    volume
-----
  1    Cone    2.00  3.00  1.60  5.00    8.04    13.40
  2    Cone    4.50  2.00  1.10  3.50    3.80    4.43
  3    Cone    3.00  5.50  2.80  2.00   24.63   16.42
  4    Cone    5.50  4.50  0.90  2.50    2.54    2.12
  5    Cone   14.00 12.00  2.40  8.00   18.10   48.25
  6    Cone   16.50 13.50  1.00  9.50    3.14    9.95
  7    Cone   15.00 15.50  1.80  6.50   10.18   22.05
  8    Cone   17.50 11.00  1.00  3.00    3.14    3.14
```

Скриншот 5. Запуск figures: разбор файла, таблица фигур, пропуск ошибочных строк

Вторая программа – figures_list: те же фигуры складываются в двунаправленный список STL и упорядочиваются сначала по площади основания, потом по объёму. Порядки получаются разными, и это видно на конусе с широким низким основанием.

```
MSYS2 - run figures_list

$ ./bin/figures_list | sed -n '1,12p;48,70p'

=====
Figures in a doubly linked list (STL std::list)
=====

--- Reading properties from 'figures.txt' ---
line 38 skipped: radius must be >= 0
line 41 skipped: need 2 to 4 numbers
objects in the list: 18

--- As created (order of the file) ---
  N    type    x      y      r      h      area     volume
10   Cone    6.00    14.00    1.50    4.50     7.07     10.60
11   Cone    2.00     3.00    1.60    5.00     8.04     13.40
12   Cone    8.00    17.00    1.70    5.50     9.08     16.65
13   Cone   15.00    15.50    1.80    6.50    10.18     22.05
14  Circle   12.50    17.50    2.00     -     12.57     -
15   Cone   19.00     4.00    2.20    9.00    15.21     45.62
16   Cone   14.00    12.00    2.40    8.00    18.10     48.25
17   Cone    3.00     5.50    2.80    2.00    24.63     16.42
18   Cone    9.00     8.00    3.00   10.00    28.27     94.25
-----
18                                total:    156.45    286.89

--- Sorted by volume ---
  N    type    x      y      r      h      area     volume
-----
 1   Point    7.50    10.50     -     -         -         -
 2   Point   13.00     7.00     -     -         -         -
 3   Point   20.00    16.00     -     -         -         -
 4  Circle   10.50     6.00    1.20     -     4.52         -
 5  Circle   11.00     3.00    1.40     -     6.16         -
 6  Circle   12.50    17.50    2.00     -    12.57         -
 7   Cone     5.50     4.50    0.90    2.50     2.54         2.12
```

Скриншот 6. Запуск figures_list: список STL и два порядка сортировки

Третья программа – points: вокруг каждого конуса она рассыпает гауссовы точки, приписывает каждой номер блока сетки, подбирает порог по минимальному покрывающему дереву и ищет кластеры двумя способами. Оба дали 4 кластера размерами 80, 100, 20 и 40 точек, а поиск ближайшей точки по блокам обошёлся 26 вычислениями расстояния вместо 240 при полном переборе – ответ тот же.

```
MSYS2 - run points

$ ./bin/points | tail -36
 4      2.87      2.37      1      1
 5      1.13      2.22      1      1
 6      2.34      2.39      1      1
 7      0.93      1.84      0      1
 8      2.35      2.72      1      1
... 240 points in total

--- Threshold from the minimum spanning tree ---
tree edges: shortest 0.01, longest 4.11
suggested threshold: 2.39

--- Binary matrix of links ---
size 240 x 240, the busiest point has 48 neighbours

--- Wave algorithm (forest fire) ---
clusters found: 4
sizes: 80 100 20 40
the fire needed 7 waves at most

--- Path algorithm (cutting a track) ---
clusters found: 4
sizes: 80 100 20 40
both algorithms found the same number of clusters

--- Nearest point to (10.70, 10.20) ---
full search : point 168, 240 distances
by blocks   : point 168, 26 distances
answers are the same, the grid checked 214 points less
GridPoint(x = 9.77, y = 10.41, block = [ 4, 5])

--- Data for gnuplot (folder 'out') ---
points.txt -- x, y, block and cluster of every point
grid_lines.txt -- lines of the block grid

Run 'gnuplot plot_points.gp' to draw the clusters.
Program finished.

$ gnuplot plot_points.gp
plots are ready: out/points_clusters.png, out/points_grid.png
```

Скриншот 7. Запуск points: порог, кластеры двумя алгоритмами, поиск по сетке

Четвёртая программа – gauss_demo: она проверяет сам датчик случайных чисел. По выборке из 5000 чисел получились среднее 0.015 и отклонение 1.014 при заданных 0 и 1, внутри одной сигмы попало 67.9 процента значений, внутри двух – 95.6 процента против теоретических 68.3 и 95.4.

```

MSYS2 - run gauss_demo
=== Горчак Дмитрий, 212. Запуск программы gauss_demo ===

$ ./bin/gauss_demo | tail -14
Gaussian distribution: one and two dimensions
=====

--- One-dimensional sample, mu = 0, sigma = 1 ---
sample mean      : 0.015
sample deviation  : 1.014
inside 1 sigma    : 67.940 % (theory 68.3 %)
inside 2 sigma    : 95.640 % (theory 95.4 %)
saved: out/gauss1d.txt (5000 numbers)
saved: out/gauss2d_rho00.txt (rho = 0.000)
saved: out/gauss2d_rho07.txt (rho = 0.700)

Run 'gnuplot plot_gauss.gp' to draw the pictures.
Program finished.

$ gnuplot plot_gauss.gp
plots are ready: out/gauss1d.png, out/gauss2d.png

$ ls -l out/gauss*
out/gauss1d.png
out/gauss1d.txt
out/gauss2d.png
out/gauss2d_rho00.txt
out/gauss2d_rho07.txt

```

Скриншот 8. Запуск gauss_demo: проверка датчика по правилам одной и двух сигм

6. Результат: поле фигур

Вид сверху: конусы, круги и точки – каждый тип своим стилем, у конусов подписана высота. Видно, что конусы собраны в две группы и несколько одиночных фигур – это заготовка для будущей задачи о кластерах.

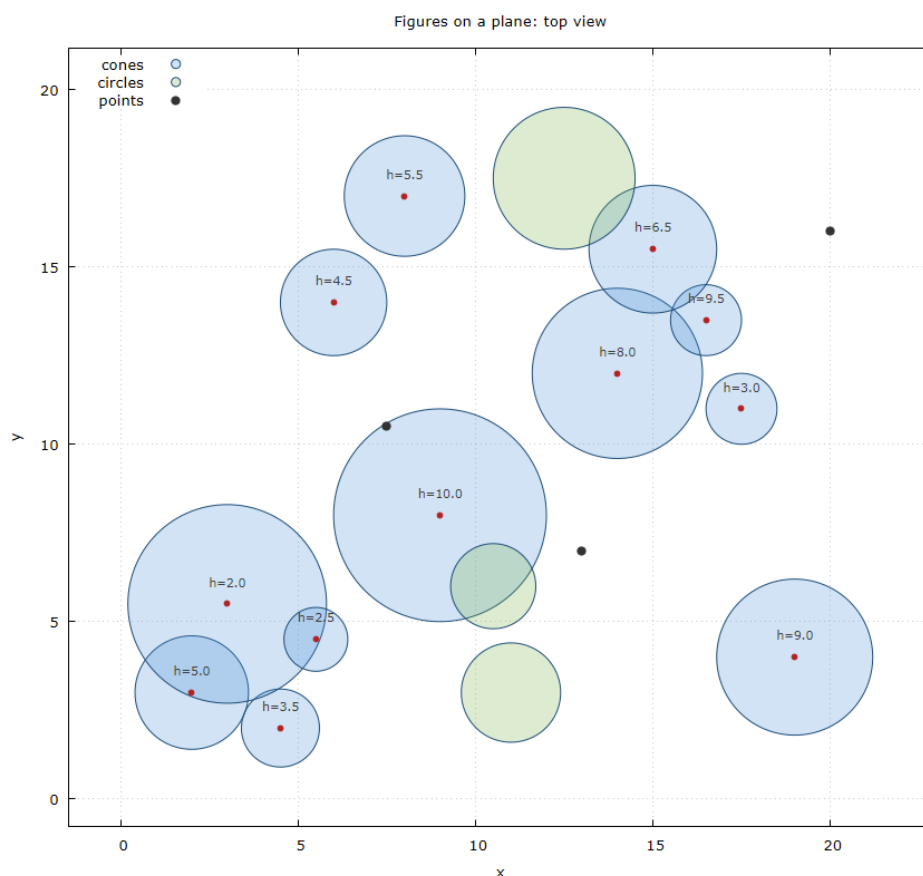


Рисунок 1. Вид сверху: конусы, круги и точки на плоскости

Ландшафт: высота поверхности в точке равна максимуму по всем конусам, поэтому там, где основания пересекаются, виден более высокий конус. Сетку 121 x 121 узел считает программа и сохраняет в файл, gnuplot только рисует.

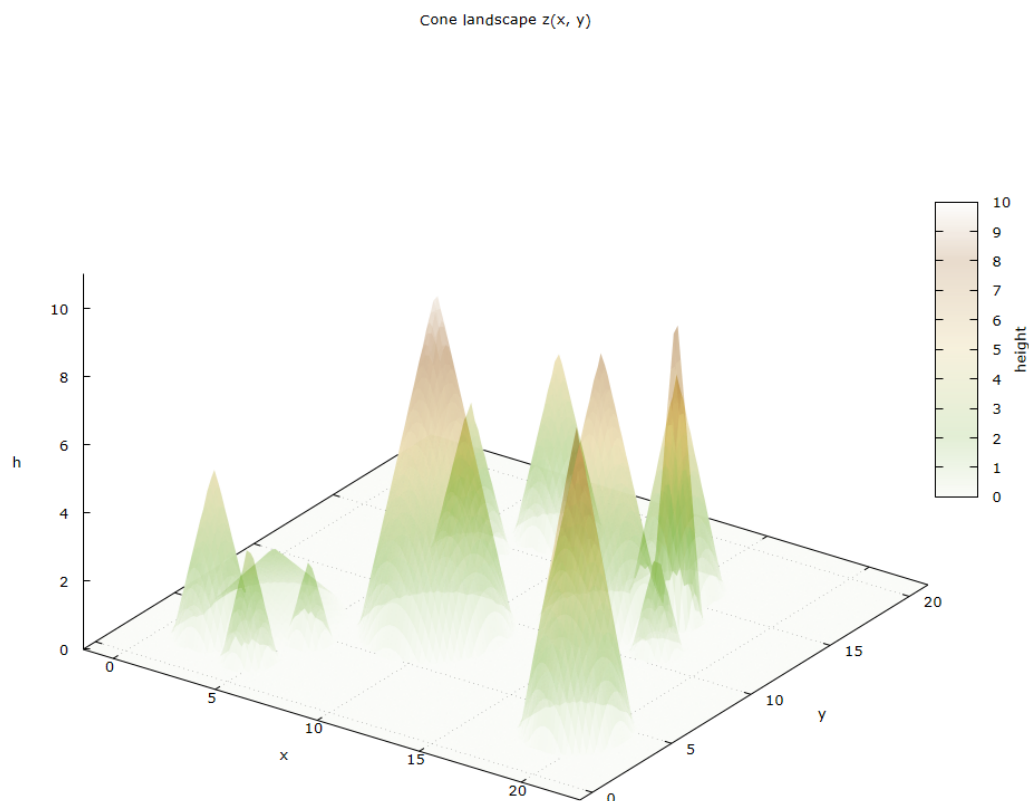


Рисунок 2. Ландшафт $z(x, y)$: поверхность конусов над плоскостью

Третий рисунок показывает фигуры точками на их поверхности. Точки расставлены не перебором пространства, а по уравнению поверхности: на высоте z радиус сечения конуса равен $r * (1 - z / h)$, и по этой окружности точки ставятся через равные углы. Уравнение каждой фигуры программа дополнительно записывает в файл `out/equations.txt`.

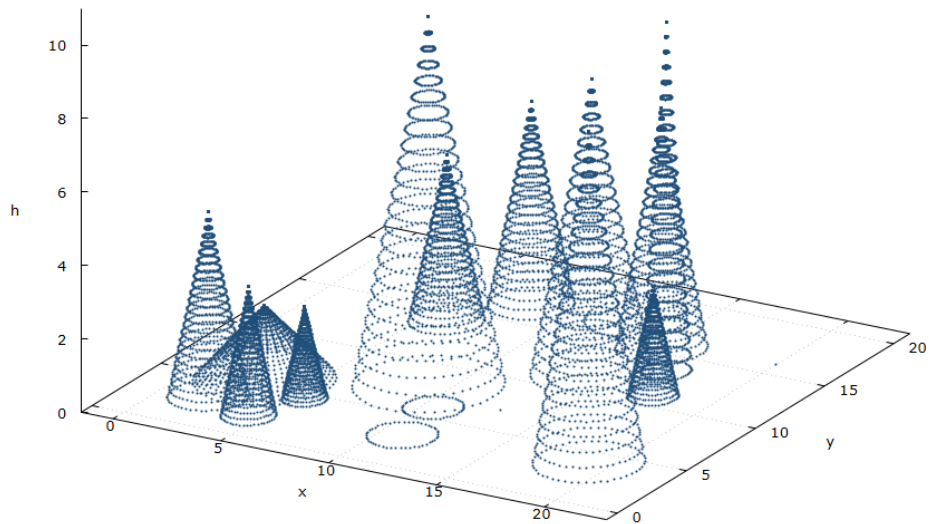


Рисунок 3. Фигуры, нарисованные точками по поверхности

7. Результат: точки, сетка и кластеры

Вокруг каждого конуса рассыпано по двадцать точек с гауссовым разбросом, равным половине радиуса основания: при таком разбросе почти все точки остаются внутри основания. Программа ничего не знает о том, какая точка от какого конуса, и восстанавливает группы только по расстояниям.

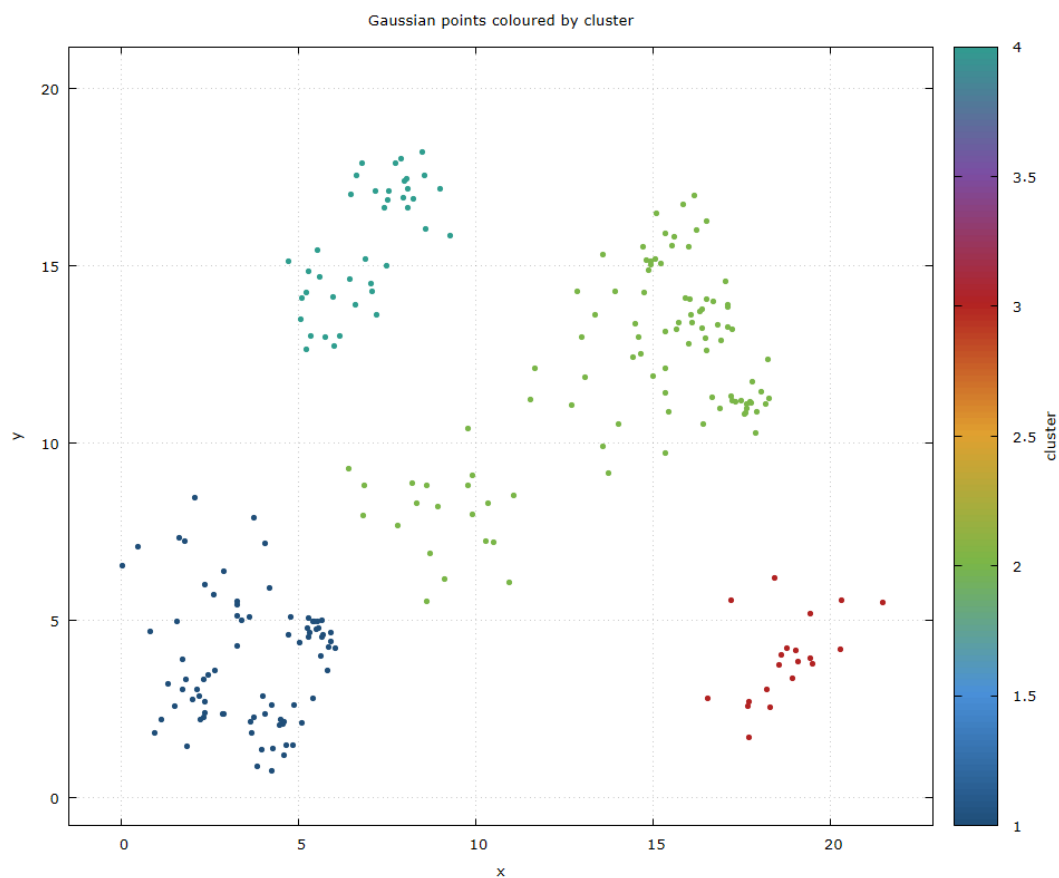


Рисунок 4. Кластеры, найденные по порогу 2.39: цвет – номер связной компоненты

Сетка блоков нужна, чтобы не перебирать всё облако при поиске соседей: по координатам точки сразу считается номер её блока, и просматриваются только сам блок и ближайшие к нему.

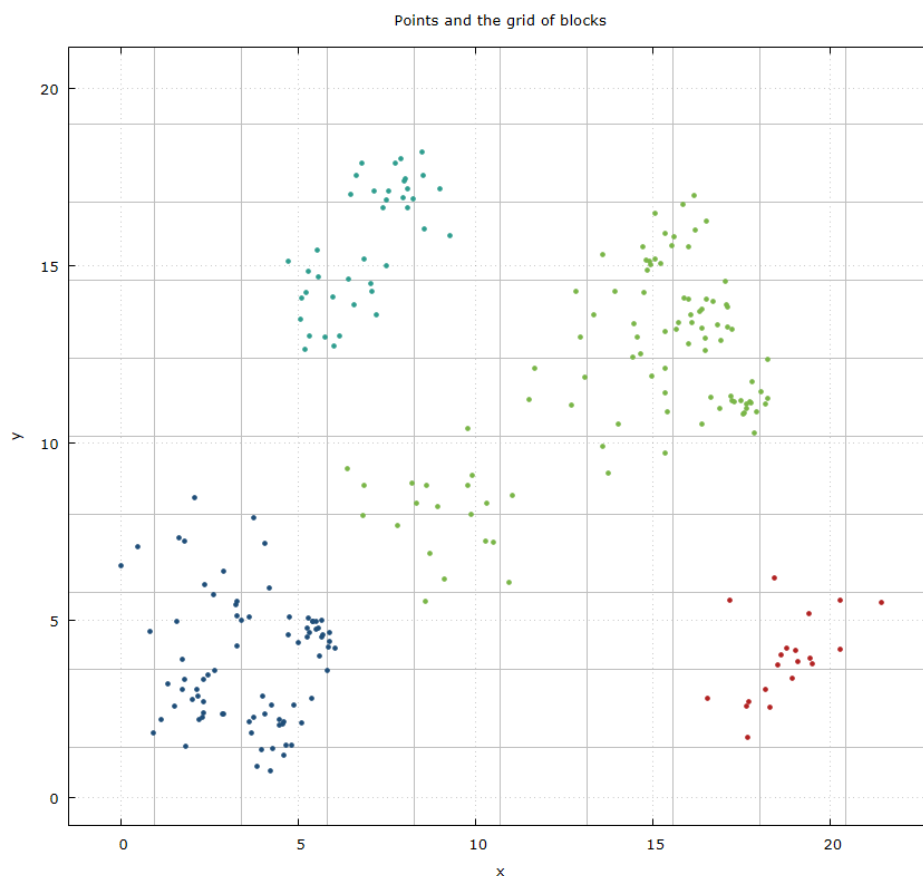


Рисунок 5. Сетка 10 x 10 блоков и распределение точек по ней

Отдельно проверен датчик случайных чисел. Гистограмма выборки ложится на теоретическую кривую плотности, а двумерное облако меняет форму при изменении одного только коэффициента связи координат: при нуле его оси совпадают с осями координат, при 0.7 облако наклонено.

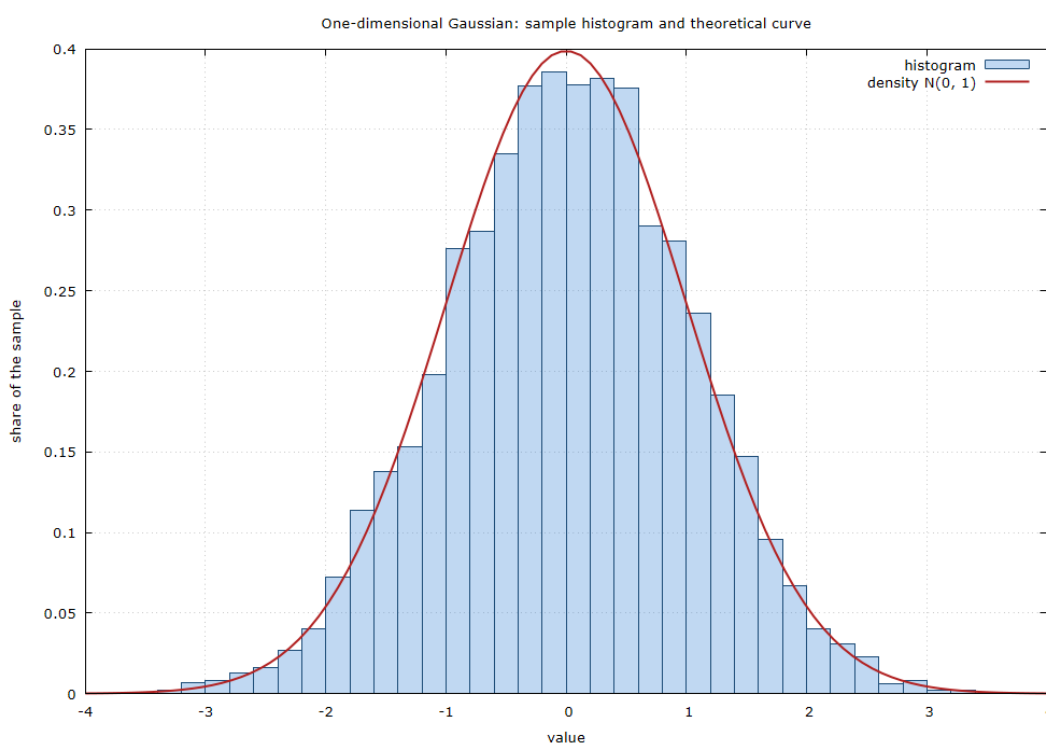
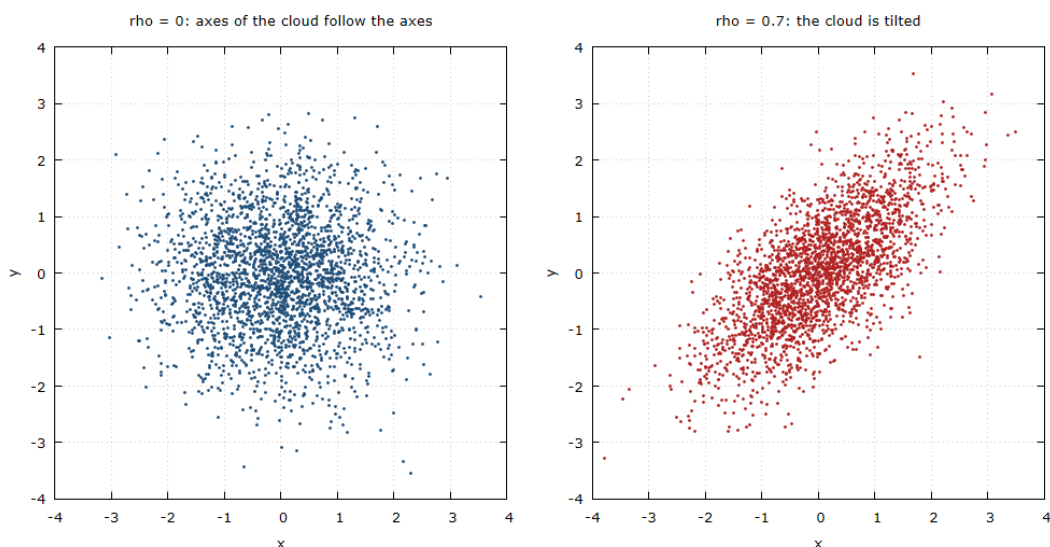


Рисунок 6. Гистограмма выборки и теоретическая кривая $N(0, 1)$

Рисунок 7. Двумерное распределение при $\rho = 0$ и $\rho = 0.7$

8. Состав репозитория

Файл	Что внутри
point.h, point.cpp	класс Point: координаты, расстояние между точками
circle.h, circle.cpp	класс Circle: радиус и площадь как метод
cone.h, cone.cpp	класс Cone: высота, объём, высота поверхности
figures_io.h, figures_io.cpp	чтение файла, таблица, данные для gnuplot
grid_point.h, .cpp	класс GridPoint: точка с номером блока сетки
grid.h, grid.cpp	сетка блоков и поиск ближайшей точки по блокам
gauss.h, gauss.cpp	класс Gauss: датчик нормальных чисел, одно и два измерения
cluster.h, cluster.cpp	матрица расстояний, порог, волновой алгоритм и просека
main.cpp	программа figures: таблица фигур и подготовка рисунков
list_demo.cpp	программа figures_list: двунаправленный список STL
points_demo.cpp	программа points: гауссовы точки, сетка, кластеры
gauss_demo.cpp	программа gauss_demo: проверка датчика случайных чисел
figures.txt	исходные данные: от двух до четырёх чисел в строке
plot_figures.gp	скрипт gnuplot: четыре рисунка по фигурам
plot_points.gp	скрипт gnuplot: кластеры и сетка блоков
plot_gauss.gp	скрипт gnuplot: гистограмма и двумерные облака
build.sh, make_txt.sh	сборка и копии исходников в .txt
README.md	описание задачи, состава и порядка запуска
essays/	эссе, этот отчёт, скриншоты и рисунки
txt/	копии всех исходников с расширением .txt

Все исходники лежат в одной папке `~/cones` без вложенных каталогов; отдельные папки заведены только под материалы отчёта (`essays`) и под копии исходников в текстовом виде (`txt`). Собирается всё одной командой `bash`

build.sh.

9. Что дальше

- 1) выложить репозиторий в облако и прислать ссылку: локальная история переносится командами `git remote add` и `git push`, сами команды работы не меняются;
- 2) ускорить построение матрицы связей: сейчас она строится полным перебором пар, хотя размер блока можно взять равным порогу и проверять только соседние блоки;
- 3) сравнить порог, подобранный по покрывающему дереву, с порогами, заданными вручную, и посмотреть, при каком значении кластеры начинают сливаться.